

Differential Geometry Topics 1

Principal directions and curvature lines (what Math3D is drawing)

Let $S \subset \mathbb{R}^3$ be a smooth regular surface and let $p \in S$.

1) Normal, shape operator, principal curvatures

Choose a (locally defined) unit normal field n along S (possible on any orientable patch). The *shape operator* (a.k.a. Weingarten map) at p is the linear map

$$S_p : T_p S \rightarrow T_p S, \quad S_p(v) = -D_v n,$$

where $D_v n$ denotes the directional derivative of the normal field in the direction v (in $\mathbb{R}^3$). Geometrically: S_p measures how the normal rotates when you move tangentially.

The *principal curvatures* k_1, k_2 at p are the eigenvalues of S_p . The corresponding eigenvectors (in $T_p S$) are the *principal directions* d_1, d_2 . At a generic point (where $k_1 \neq k_2$), these directions are orthogonal and determined up to sign.

A *curvature line* is a curve $\gamma(s) \subset S$ whose tangent is everywhere aligned with a principal direction field:

$$\gamma'(s) \parallel d_1(\gamma(s)) \quad \text{or} \quad \gamma'(s) \parallel d_2(\gamma(s)).$$

Math3D viewpoint. Curvature lines are obtained by numerically integrating the direction field d_1 or d_2 over the surface, i.e. tracing streamlines of principal directions.

Implicit surface case $F(x, y, z) = 0$: principal directions at a point

Assume $F : \mathbb{R}^3 \rightarrow \mathbb{R}$ is smooth and p is a regular point:

$$\nabla F(p) \neq 0.$$

2) Normal from the gradient. Let

$$g = \nabla F(p), \quad n = \frac{g}{\|g\|}.$$

Then $T_p S = \{v \in \mathbb{R}^3 : g \cdot v = 0\}$ and n is the unit normal.

3) Using the Hessian to get a curvature operator. Let $H = \text{Hess } F(p)$ be the 3×3 Hessian matrix at p . A convenient formula for the derivative of the *unit* normal is:

$$D_v n = \frac{1}{\|g\|} H v - \frac{g^\top H v}{\|g\|^3} g, \quad (v \in \mathbb{R}^3).$$

For a tangent vector $v \in T_p S$ (so $g \cdot v = 0$), the shape operator is obtained by taking the negative tangential component:

$$S_p(v) = -\Pi_T(D_v n), \quad \Pi_T = I - nn^\top.$$

Here Π_T is the orthogonal projection onto $T_p S$. Numerically, this projection is useful because floating point errors can introduce a small normal component in $D_v n$ even when the true value is tangent.

4) Turning this into a 2×2 eigenproblem. Construct an orthonormal basis $\{t_1, t_2\}$ of $T_p S$ (e.g. pick any vector not parallel to n , take cross products, normalize). Define the 2×2 matrix

$$A_{ij} = \langle S_p(t_j), t_i \rangle, \quad i, j \in \{1, 2\}.$$

Then:

- the eigenvalues of A are the principal curvatures k_1, k_2 ,
- an eigenvector (a, b) of A corresponds to a principal direction

$$d = a t_1 + b t_2 \in T_p S.$$

This is precisely the *per-point principal direction field* that a curvature-line integrator traces.

Cone analysis (key example): what principal lines should look like

Consider a right circular cone without the apex, with slope $m > 0$:

$$z = mr, \quad r = \sqrt{x^2 + y^2}.$$

Equivalently, in implicit form (away from the apex):

$$F(x, y, z) = x^2 + y^2 - \frac{1}{m^2} z^2 = 0.$$

5) A clean parametrization

A standard parametrization is

$$\phi(u, v) = (u \cos v, u \sin v, mu), \quad u > 0, \quad v \in [0, 2\pi).$$

Here u is the distance from the axis (radius) and v is the angular coordinate. The tangent vectors are

$$\phi_u = (\cos v, \sin v, m), \quad \phi_v = (-u \sin v, u \cos v, 0).$$

6) Crucial geometric fact: the normal does not depend on u

Compute (up to orientation) a unit normal:

$$n(v) = \frac{1}{\sqrt{1+m^2}}(-m \cos v, -m \sin v, 1).$$

Notably, n depends only on v , not on u . Therefore moving along a generator (the u -direction) does not change the normal:

$$D_{\phi_u} n = 0 \implies k_{\text{generator}} = 0.$$

Hence one principal curvature vanishes everywhere on the cone (except at the apex, which is not regular).

7) Principal curvatures and directions on the cone

In these coordinates the principal directions align with parameter curves:

Principal direction 1 (zero curvature): along generators.

$$d_1 \parallel \phi_u = (\cos v, \sin v, m), \quad k_1 = 0.$$

Principal direction 2 (nonzero curvature): around the cone ($u = \text{const}$).

$$d_2 \parallel \phi_v = (-u \sin v, u \cos v, 0), \quad k_2 = \frac{m}{u(1+m^2)}.$$

If α is the angle between the generator and the axis, then $m = \cot \alpha$, and

$$k_2 = \frac{\cos \alpha}{u}.$$

Consequences.

$K = k_1 k_2 = 0$ (developable surface away from the apex), $k_2 \sim \frac{1}{u}$ blows up as $u \rightarrow 0$.

The apex is singular (not a regular surface point).

8) Expected curvature-line picture on the cone

- Curvature lines for d_1 : straight rulings (generators), i.e. straight lines on the cone.
- Curvature lines for d_2 : circles $u = \text{const}$, i.e. intersections with planes $z = \text{const}$.

So if Math3D integrates principal directions on a cone: one family should appear as straight lines running along the cone, and the other family should wrap around as rings.

Ridges and valleys on surfaces (general theory)

Let $S \subset \mathbb{R}^3$ be a smooth oriented surface and let $p \in S$ be a regular point. Choose a unit normal field n (locally). The *shape operator* (Weingarten map) is

$$S_p : T_p S \rightarrow T_p S, \quad S_p(v) = -D_v n,$$

and its eigenvalues $k_1(p) \geq k_2(p)$ are the *principal curvatures* with corresponding unit eigenvectors (principal directions) $d_1(p), d_2(p) \in T_p S$.

A *curvature line* is a curve $\gamma(s) \subset S$ whose tangent satisfies

$$\gamma'(s) \parallel d_1(\gamma(s)) \quad \text{or} \quad \gamma'(s) \parallel d_2(\gamma(s)).$$

Directional derivative on the surface. For a scalar field f on S define the directional derivative along a tangent unit vector $v \in T_p S$ by

$$\partial_v f(p) := \nabla_S f(p) \cdot v,$$

where ∇_S denotes the surface gradient.

Ridge/valley conditions (canonical). A k_1 -*ridge* is the locus where k_1 has a local *maximum* along its own principal direction:

$$\partial_{d_1} k_1 = 0, \quad \partial_{d_1}^2 k_1 < 0.$$

A k_2 -*valley* is the locus where k_2 has a local *minimum* along its own principal direction:

$$\partial_{d_2} k_2 = 0, \quad \partial_{d_2}^2 k_2 > 0.$$

(Other conventions also consider k_1 -valleys or k_2 -ridges by flipping the second-derivative sign.)

Practical stability notes. At *umbilics* ($k_1 \approx k_2$) the principal directions are ill-conditioned; on nearly flat regions ($|k_i| \approx 0$) derivative estimates are noise-dominated. Numerically one typically gates detection by

$$|k_1 - k_2| > \varepsilon \quad \text{and/or} \quad |k_i| > \kappa_{\min},$$

together with additional smoothing or robust finite-difference schemes.

Ridges and valleys for an implicit surface $F(x, y, z) = 0$

Assume $F : \mathbb{R}^3 \rightarrow \mathbb{R}$ is smooth and $p \in S = \{F = 0\}$ is regular, i.e. $\nabla F(p) \neq 0$. Let

$$g = \nabla F(p), \quad n = \frac{g}{\|g\|}, \quad H = \text{Hess } F(p).$$

For a tangent vector $v \in T_p S$ (so $g \cdot v = 0$), a useful identity is

$$D_v n = \frac{1}{\|g\|} H v - \frac{g^\top H v}{\|g\|^3} g.$$

The shape operator is then obtained by projecting back to the tangent plane:

$$S_p(v) = -\Pi_T(D_v n), \quad \Pi_T = I - nn^\top.$$

To compute (k_i, d_i) numerically, build an orthonormal tangent basis (t_1, t_2) at p and form the 2×2 matrix

$$A_{ij} = \langle S_p(t_j), t_i \rangle, \quad i, j \in \{1, 2\}.$$

Eigenpairs of A give the principal curvatures and directions.

Ridge/valley derivatives. Exact evaluation of $\partial_{d_i} k_i$ involves third derivatives of F (since k_i depends on H). A robust alternative is a *finite difference on the surface*:

$$\partial_{d_i} k_i(p) \approx \frac{k_i(p_+) - k_i(p_-)}{2h},$$

where $p_{\pm}$ are obtained by stepping in $\mathbb{R}^3$ and projecting back to $F = 0$:

$$\tilde{p}_{\pm} = p \pm h d_i(p), \quad p_{\pm} \approx \text{ProjectToSurface}(\tilde{p}_{\pm}).$$

A simple projection step uses Newton correction along the gradient:

$$q \leftarrow q - \frac{F(q)}{\|\nabla F(q)\|^2} \nabla F(q),$$

applied once or twice.

Ridges and valleys for an explicit graph $z = f(x, y)$

For the graph surface $\phi(x, y) = (x, y, f(x, y))$, the unit normal is

$$n = \frac{(-f_x, -f_y, 1)}{\sqrt{1 + f_x^2 + f_y^2}}.$$

Principal curvatures and directions can be computed from the first and second fundamental forms in terms of $f_x, f_y, f_{xx}, f_{xy}, f_{yy}$ (standard differential geometry formulas).

Ridge/valley derivatives. In analytic form, ridge tests involve third derivatives of f . In practice one may compute k_i on a grid and approximate

$$\partial_{d_i} k_i \approx \frac{k_i(x_+, y_+) - k_i(x_-, y_-)}{2h},$$

where $(x_{\pm}, y_{\pm}) = (x, y) \pm h \hat{d}_i$ and $\hat{d}_i$ is the principal direction represented in the parameter plane. (No surface projection is needed for graphs.)

Ridges and valleys for a parametric surface $\phi(u, v)$

Let $\phi : U \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$ be a regular parametrization. Compute first derivatives ϕ_u, ϕ_v and second derivatives $\phi_{uu}, \phi_{uv}, \phi_{vv}$. The first fundamental form is

$$E = \langle \phi_u, \phi_u \rangle, \quad F = \langle \phi_u, \phi_v \rangle, \quad G = \langle \phi_v, \phi_v \rangle, \quad I = \begin{pmatrix} E & F \\ F & G \end{pmatrix}.$$

The unit normal is

$$n = \frac{\phi_u \times \phi_v}{\|\phi_u \times \phi_v\|}.$$

The second fundamental form coefficients are

$$L = \langle \phi_{uu}, n \rangle, \quad M = \langle \phi_{uv}, n \rangle, \quad N = \langle \phi_{vv}, n \rangle, \quad II = \begin{pmatrix} L & M \\ M & N \end{pmatrix}.$$

The shape operator matrix in the (ϕ_u, ϕ_v) basis is

$$S = I^{-1}II.$$

Eigenpairs of S give principal curvatures k_i and principal directions in parameter coordinates, which can be mapped to tangent vectors in $\mathbb{R}^3$.

Ridge/valley derivatives (practical). Exact formulas require derivatives of S (hence third derivatives of ϕ and connection terms). A robust viewer-friendly approach is finite differencing in parameter space along principal directions:

$$\partial_{d_i} k_i(u, v) \approx \frac{k_i(u_+, v_+) - k_i(u_-, v_-)}{2h},$$

with $(u_{\pm}, v_{\pm}) = (u, v) \pm h \hat{d}_i$ where $\hat{d}_i$ is the principal direction represented in (u, v) .

Ridges and valleys for minimal surfaces (Weierstrass representation)

Let a minimal surface be given by Weierstrass data (g, ω) (meromorphic g and holomorphic 1-form ω), with immersion

$$\phi(z) = \Re \int \left(\frac{1}{2}(1 - g^2)\omega, \frac{i}{2}(1 + g^2)\omega, g\omega \right).$$

Minimality implies mean curvature $H = 0$, hence

$$k_1 = -k_2.$$

Therefore ridge/valley structure is symmetric, and it is often more robust to study extrema of the curvature magnitude

$$|k| := |k_1| = |k_2|,$$

rather than treating two unrelated families.

Practical computation. In an interactive tool one typically samples $\phi(u, v)$ and then applies the *parametric* pipeline above to obtain (k_i, d_i) and approximate $\partial_{d_i} k_i$ (or $\partial_{d_i} |k|$) by finite differences.

Ridges and valleys on a triangle mesh (overview)

Input (per mesh vertex v).

$$p_v \in \mathbb{R}^3, \quad n_v \in \mathbb{S}^2, \quad k_1(v) \geq k_2(v), \quad d_1(v), d_2(v) \in T_v S, \quad \|d_i(v)\| = 1.$$

Output. Polyline approximating *ridges* and *valleys* (with a type label), robustly clipped near umbilics and boundaries.

Key idea. Build a signed scalar field

$$g_1(v) = \partial_{d_1} k_1(v) = \nabla_S k_1(v) \cdot d_1(v),$$

trace the zero set $g_1 = 0$ on the mesh (triangle-by-triangle), then keep only those points where k_1 is a *true extremum* along the d_1 direction (max $\Rightarrow$ ridge, min $\Rightarrow$ valley).

Step 0 — knobs that prevent explosions (practical gating)

Geometry gating (umbilics / flat regions). *Anisotropy gate (umbilic avoidance):*

$$A(v) = \frac{|k_1(v) - k_2(v)|}{|k_1(v)| + |k_2(v)| + \varepsilon}, \quad \text{require } A(v) \geq A_{\min},$$

with typical $A_{\min} \in [0.15, 0.30]$.

Curvature magnitude gate:

$$|k_1(v)| \geq k_{\min},$$

typical choices: $k_{\min} \approx 0.5/R$ for an expected feature radius R , or in mesh units $k_{\min} \approx 0.5/\text{avgEdgeLen}$.

Derivative stability. *Gradient magnitude gate:*

$$\|\nabla_S k_1(v)\| \geq \text{grad}_{\min}, \quad \text{grad}_{\min} \approx 0.25 \frac{|k_1(v)|}{\text{avgEdgeLen}}.$$

Boundary avoidance. Mask triangles within `boundary_rings` of the boundary (typical `boundary_rings` = 1 or 2).

Polyline cleanup.

$$\text{min_polyline_length} = 2 \text{ avgEdgeLen}, \quad \text{max_segments_per_polyline} = 20000,$$

merge endpoints with

$$\text{merge_eps} = 10^{-4} \text{ bboxDiag} \quad (\text{or } 10^{-3} \text{ avgEdgeLen}).$$

Optional smoothing (highly recommended). Smooth k_1, k_2 for `smooth_iters` = 1..5 (cotan Laplacian or umbrella), then recompute $\nabla_S k_i$ and g_i . Keep `smooth_lambda` small (0.1..0.3).

Step 1 — compute $\nabla_S k_1$ per vertex (robust least squares)

Compute $\nabla_S k_1(v)$ as a *tangent* vector to avoid line-field sign issues.

1.1 Tangent frame at v . Build an orthonormal frame (t_u, t_v) spanning $T_v S$: pick any vector a not parallel to n_v , then

$$t_u = \text{normalize}(n_v \times a), \quad t_v = n_v \times t_u.$$

1.2 Weighted least squares in the tangent plane. For each neighbor $w \in N(v)$:
edge vector

$$e = p_w - p_v,$$

project to the tangent plane at v :

$$e_T = e - (e \cdot n_v) n_v,$$

2D coordinates in the tangent frame:

$$x = e_T \cdot t_u, \quad y = e_T \cdot t_v,$$

scalar difference:

$$\Delta k = k_1(w) - k_1(v).$$

Solve for (g_x, g_y) in

$$\Delta k \approx g_x x + g_y y$$

using weights, e.g.

$$\omega = \frac{1}{\|e\|^2},$$

and ignore neighbors with $\|e_T\| < \eta$ (tiny tangential displacement).

1.3 Convert back to a 3D tangent gradient.

$$\nabla_S k_1(v) = g_x t_u + g_y t_v.$$

(Do the same for k_2 if you also want $g_2 = \partial_{d_2} k_2$.)

Step 2 — define the signed scalar field g_1

At each vertex,

$$g_1(v) = \nabla_S k_1(v) \cdot d_1(v).$$

Important: line-field sign issue (fix locally per triangle). Principal directions are a *line field*: d and $-d$ represent the same geometric direction. Therefore g_1 can flip sign arbitrarily if d_1 flips between vertices.

Do *local per-triangle* alignment during tracing:

For a triangle with vertices (a, b, c) , choose a reference direction d_a . Align d_b, d_c to it:

$$\text{if } d_b \cdot d_a < 0, \text{ set } d_b \leftarrow -d_b, \quad \text{if } d_c \cdot d_a < 0, \text{ set } d_c \leftarrow -d_c.$$

Then compute triangle-consistent scalars

$$g_a = \nabla_S k_1(a) \cdot d_a, \quad g_b = \nabla_S k_1(b) \cdot d_b, \quad g_c = \nabla_S k_1(c) \cdot d_c.$$

Do *not* attempt to globally orient d_1 on the whole mesh; local per-triangle alignment is the robust strategy.

Step 3 — trace the zero set $g_1 = 0$ (edge-walk / marching triangles)

This is marching triangles implemented as an edge-walk that produces polylines.

3.1 Per triangle: find crossings on edges

For each triangle (a, b, c) with triangle-consistent values (g_a, g_b, g_c) :

For each edge (u, v) among (a, b) , (b, c) , (c, a) : if $g_u g_v < 0$ (opposite signs) and endpoints pass the masks (anisotropy, curvature, boundary, ...), then the isoline crosses that edge.

Interpolate the crossing parameter:

$$t = \frac{g_u}{g_u - g_v} \in (0, 1),$$

and the crossing point in $\mathbb{R}^3$:

$$x = (1 - t)p_u + tp_v.$$

Degeneracies. If $|g|$ is tiny at a vertex, clamp with an epsilon policy to avoid noisy “three-edge” cases. A common choice is: if $|g_u| < g_\varepsilon$, replace g_u by $\text{sign}(g_u) g_\varepsilon$ unless you explicitly want vertex crossings.

3.2 In-triangle segment creation

In a generic triangle the isoline crosses either 0 edges (no segment) or 2 edges (one segment). If you found two crossings x_0, x_1 , create one segment

$$\text{segment} = (x_0, x_1, \text{tri_id}, (\text{edge}_0, \text{edge}_1)).$$

3.3 Stitch segments into polylines (edge-walk)

You now have short segments (typically one per intersected triangle). Stitch them into polylines by matching segment endpoints across shared mesh edges:

- Key each crossing by a stable edge identity (undirected mesh edge id) plus a quantized t , or store an explicit “edge-crossing record” that adjacent triangles can reference.
- Build adjacency: each crossing on an interior edge has exactly one matching crossing from the adjacent triangle.
- Start from any unused segment and follow adjacency endpoint-to-endpoint, appending points, until:
 - you hit a boundary edge (no adjacent triangle) $\Rightarrow$ open polyline end,
 - you return to the start $\Rightarrow$ closed loop,
 - you exceed `max_segments_per_polyline` $\Rightarrow$ stop (safety cap).

Step 4 — classify each polyline as ridge vs valley (stable extremum test)

The set $g_1 = 0$ contains both maxima and minima candidates of k_1 along d_1 . We keep only points that are genuine extrema using a robust discrete proxy.

4.1 Interpolate d_1 and k_1 at a polyline point

For a point x on a segment inside triangle $T = (a, b, c)$, compute barycentric weights $(\lambda_a, \lambda_b, \lambda_c)$.

Using triangle-aligned directions d_a, d_b, d_c , define

$$d(x) = \text{normalize}(\lambda_a d_a + \lambda_b d_b + \lambda_c d_c),$$

and interpolate curvature

$$k(x) = \lambda_a k_1(a) + \lambda_b k_1(b) + \lambda_c k_1(c).$$

4.2 Sample k_1 at offsets $\pm h$ along $d(x)$ and project back to the mesh

Choose a step

$$h = h_{\text{factor}} \cdot \text{avgEdgeLen}, \quad h_{\text{factor}} \in [0.75, 1.5].$$

Form offset targets in $\mathbb{R}^3$:

$$x_{\pm}^{\text{target}} = x \pm h d(x).$$

Project each target back to the mesh by a robust local method (nearest-triangle / vertex-walk): start from the current triangle, hop across edges if the closest point lies outside, and stop after `proj_max_steps` (e.g. 20). This mirrors a streamline-style projection step.

Let the projected points be x_+ and x_- , and interpolate k_1 there to get k_+ and k_- .

4.3 Extremum test (ridge vs valley)

Use a noise margin

$$\delta_k = \delta_{\text{factor}} \cdot |k(x)|, \quad \delta_{\text{factor}} \in [0.02, 0.10].$$

Ridge (local maximum of k_1 along d_1):

$$k(x) > k_+ + \delta_k \quad \text{and} \quad k(x) > k_- + \delta_k.$$

Valley (local minimum of k_1 along d_1):

$$k(x) < k_+ - \delta_k \quad \text{and} \quad k(x) < k_- - \delta_k.$$

Keep only ridge points for ridge polylines and valley points for valley polylines. Optionally split a polyline where the classification changes.

Remark. This replaces a strict second-derivative condition (e.g. $\partial_{d_1}^2 k_1 < 0$ for ridges) with a stable discrete test based on sampling and projection.

Step 5 — final cleanup (must-have for stability)

After classification:

- Drop polylines shorter than `min_polyline_length`.
- Optionally simplify (Douglas–Peucker) with

$$\text{simp_eps} = 0.25 \cdot \text{avgEdgeLen}.$$

- Optionally resample to near-uniform spacing for rendering.